From Separation Logic to a Checked Cyclic Certificate

A Theory-to-Code Tutorial for `Rocq-copy/SP`

Cyclist Rocq project documentation

Source revision
`08f6cd8834a3fc22088a064025abb9ee65c7f545`

Purpose and reader contract

This tutorial is for a separation-logic researcher who already reads Rocq but wants to understand exactly what theorem the `SP` implementation proves, how executable search reaches that theorem, and where the current prover stops. The exposition proceeds from mathematics to representation to trusted theorem. It does not teach Gallina syntax, and it does not infer features from `SP/IMPLEMENTATION_PLAN.md` that are absent from the checked source.

The implementation is small enough to audit—twelve `SP` modules and two generic `Cyclic` modules—but it crosses several conceptual boundaries: finite-heap semantics, finite approximants of inductive predicates, local rule soundness, finite cyclic graphs, global trace progress, reflection, and bounded depth-first search. The main discipline of this document is therefore:

> Every executable success is followed to a kernel-checked semantic theorem; every executable failure is described only as bounded non-success.

Selected excerpts reproduce the revision named on the title page. A source reference such as `predicate_holds_rank` in `SP/Semantics.v` names both the module and the symbol to inspect. The source remains authoritative.

Contents

Purpose and reader contract i

1 Orientation: the theorem-producing path 1

2 Reified symbolic heaps 3

3 Finite-heap and ranked semantics 5
3.1 Finite partial heaps and separating conjunction . . . 5
3.2 Finite approximants of inductive predicates . . . 6
3.3 Disjunction, existential witnesses, tags, and validity . . . 7

4 Definition libraries and benchmark judgments 9

5 The generic cyclic framework 11
5.1 Local rules as validity and countermodel propagation . . . 11
5.2 Proof graphs and structural well-formedness . . . 12
5.3 Relational closure and the global progress check . . . 13
5.4 Finite trace witnesses . . . 14
5.5 Certificates and lexicographic infinite descent . . . 15

6 The implemented separation-logic rules 17
6.1 Identity and tag erasure . . . 17
6.2 Ex falso from an explicit contradiction . . . 18
6.3 Right introduction of an inductive predicate . . . 18
6.4 Left disjunction and right choice . . . 19
6.5 Exact backlinks and the teaching-only progressing loop . . . 19

7 Executable rule generation 21
7.1 One-way ordered matching . . . 21
7.2 Candidate priority and conservative stubs . . . 22

8 Reflection: from a checked graph to semantic evidence 24
8.1 Why reflection is needed . . . 24
8.2 Reflecting the finite witness . . . 25
8.3 Acyclic leaves use the same certificate interface . . . 26

9 Graph exploration and the public search API **28**
9.1 First-open-node depth-first search . . . 28
9.2 Witness enumeration and closed-graph checking . . . 29
9.3 Iterative deepening and the meaning of `solvedb` . . . 30

10 Complete vertical slice: `benchmark_base_19_1` **31**
10.1 The mathematical entailment . . . 31

11 Examples, tests, and automation utilities **33**
11.1 Accepted and rejected teaching graphs . . . 33
11.2 Regression tests for generators . . . 34
11.3 Verified automation utilities . . . 34

12 Limitations and precise claims **35**

13 Trust boundary and assumptions **37**

14 A theorem-first extension checklist **39**

A Mathematics-to-Rocq correspondence **41**

B Module and symbol index **43**

C End-to-end audit checklist **45**

Conclusion **46**

Chapter 1

Orientation: the theorem-producing path

Mathematical intention

An automated entailment prover should not merely return "yes." Its useful output is a derivation object whose acceptance implies semantic validity. For this project, the complete path is

$$\boxed{\text{judgment}} \longrightarrow \boxed{\text{generator}} \longrightarrow \boxed{\text{proof graph}} \longrightarrow \boxed{\text{checks}} \longrightarrow \boxed{\text{certificate}} \longrightarrow \boxed{\text{semantic validity}}.$$

Each arrow changes the kind of evidence. A judgment is reified syntax. A generator may return only theorem-backed rule packages. Search connects those packages into a finite graph. Boolean checks validate finite structure and progress data. Reflection turns a successful equation into dependent semantic evidence. Finally, a generic well-founded argument rules out a countermodel at the root.

This architecture follows the separation between an object logic and a generic cyclic engine explored in cyclic proof theory and in Cyclist [3, 4]. The object logic must say what a world is, what a judgment means, and what measure a trace follows. The generic layer need not know what a heap or an inductive predicate is.

Rocq realization

Figure 1.1 is the module map. Solid arrows mean a direct logical dependency in the relevant build path. Dashed arrows show clients used for examples or regression tests. Every `SP/*.v` module appears.

At the public boundary, `search_sound` in `SP/Search.v` has the form

$$\mathsf{solvedb}(c, J) = \mathsf{true} \quad \Longrightarrow \quad \mathsf{Valid}(J).$$

Here $\mathsf{Valid}(J)$ is not another Boolean: it abbreviates $\forall w, [\![J]\!](w)$. Chapter 9 traces the implementation of this implication.

Soundness and limitations

The direction is intentionally one-way. Successful search provides a proof; unsuccessful bounded search provides no countermodel and establishes no negation. Moreover, the path is proof-producing only because generators return records containing proofs. A graph node never receives an unproved rule name that the checker later interprets on trust.

The `Cyclic` layer also runs two notions of global progress: a relational closure check and a concrete finite trace-witness check. Both are required for acceptance, but only the witness is projected into the semantic certificate. This deliberate distinction is developed in Chapters 5 and 8.

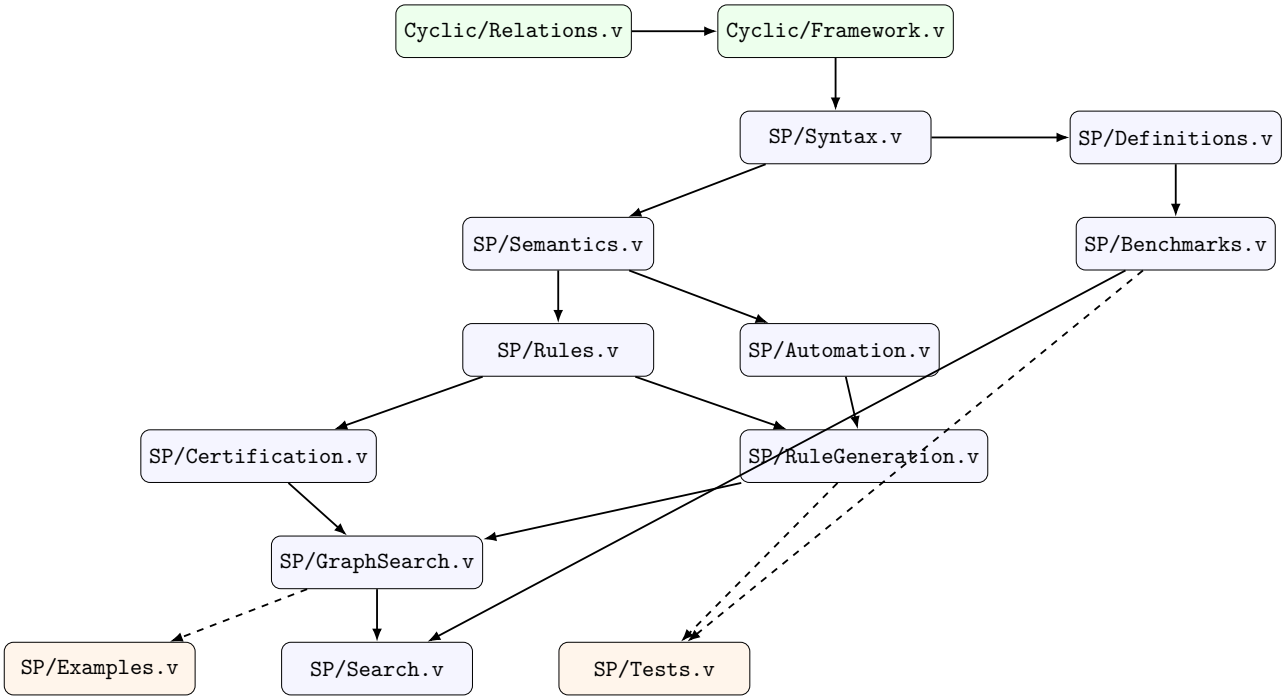


Figure 1.1: Complete module map for the documented `SP` implementation and the required generic `Cyclic` substrate.

Chapter 2

Reified symbolic heaps

Mathematical intention

Fix a countable set of locations, with a distinguished null location. Terms are either null or variables. Variables have two roles: free variables obtain their value from the ambient world, while existential variables obtain their value from a witness chosen when a formula is satisfied.

A symbolic heap has the conventional shape

$$\Pi \ \wedge \ \Sigma, \qquad \Sigma = A_1 * \cdots * A_n,$$

where Π is a conjunction of equalities and disequalities, and each spatial atom is either a points-to cell or an inductive predicate occurrence. This implementation stores points-to atoms and predicate atoms in separate lists. A formula is

$$C \ \wedge \ (H_1 \vee \cdots \vee H_k),$$

where C is a conjunction of constraints between trace-tag bounds. The empty list of disjuncts denotes false; the one-element formula containing an atom-free symbolic heap denotes *emp*.

Tags are proof annotations on predicate occurrences, not part of predicate membership. A tag is anonymous, free, or existential. Named tags may occur in strict or non-strict ordinal constraints. An inductive definition maps a predicate name to clauses $B(\bar{x}, \bar{z}) \Rightarrow P(\bar{x})$, with the extra variables $\bar{z}$ local to a clause instance.

Rocq realization

The central records in `SP/Syntax.v` preserve these layers rather than flattening them:

```
Record symheap : Type := {
  heap_pure : list pure_atom;
  heap_points : list points_to;
  heap_predicates : list tagged_predicate
}.

Record formula : Type := {
  formula_constraints : list ordinal_constraint;
  formula_heaps : list symheap
}.

Record sequent : Type := {
  antecedent : formula;
```

```
  succedent : formula
}.
```

Listing 2.1: The reified symbolic-heap core.

The excerpt realizes the mathematical shape above: list conjunction is used inside a heap and list disjunction at formula level. Definitions are reified as `list (string * list clause)`, and a `judgment` packages the definition environment together with its sequent. Consequently equality of judgments includes equality of the entire definition environment.

All syntax has decidable structural equality. In particular, `judgment_eqb_spec` in `SP/Syntax.v` reflects a successful Boolean comparison to propositional equality. The graph builder relies on this exact equality when installing an axiom or rule. `backlink_eqb` in `SP/Syntax.v` is different: it first erases predicate tags and formula constraints but retains definitions, terms, predicates, points-to atoms, and disjunct structure. This relaxed test does not itself justify a backlink; the backlink still carries a theorem-backed unary rule.

Term substitutions are association lists. `unify_term` in `SP/Syntax.v` is a one-way matcher: `Nil` matches only `Nil`, whereas any pattern `Var x` may be bound to the target term. A repeated variable must receive the same target. Tag substitutions are separate, so a term match can never silently change trace annotations. Freshening shifts only existential identifiers; free parameters retain their identity.

The visible trace tags of a judgment are computed by `judgment_tags` in `SP/Syntax.v`. Only `FreeTag n` occurrences in the antecedent are exposed. Anonymous and existential tags are not framework trace identifiers.

Soundness and limitations

The representation is deliberately intensional. Spatial atoms are not sorted, equalities are not solved, and disjuncts are not canonicalized. At this revision, all four normalization functions are the identity. Thus two mathematically equivalent heaps need not be structurally equal or match the same generator. This incompleteness is safe: structural equality is used to restrict proof construction, never to prove semantic equivalence.

The substitution code is intentionally small. It performs a single lookup rather than recursively normalizing substitution ranges. The later candidate comparison against the complete goal is therefore important: a partially matching or differently represented instance is rejected rather than treated as a theorem. The syntax supports more transformations than search currently uses; unused support must not be confused with an implemented inference rule.

Chapter 3

Finite-heap and ranked semantics

3.1 Finite partial heaps and separating conjunction

Mathematical intention

A concrete heap is a finite partial map from non-null locations to finite tuples of locations. Write $h_1 \perp h_2$ when their domains are disjoint and $h = h_1 \uplus h_2$ for their disjoint union. Exact points-to ownership is

$$\rho, h \models t \mapsto \bar{u} \quad \Longleftrightarrow \quad [\![t]\!]_\rho \neq 0 \ \wedge \ h = [([\![t]\!]_\rho, [\![\bar{u}]\!]_\rho)].$$

The heap is exactly the singleton cell; this is not an "at least this cell" interpretation.

The semantic clause for separating conjunction is the familiar one [1]:

$$h \models P * Q \quad \Longleftrightarrow \quad \exists h_1, h_2.\ h = h_1 \uplus h_2 \wedge h_1 \models P \wedge h_2 \models Q. \tag{3.1}$$

An empty spatial conjunction owns the empty heap. Iterating Equation (3.1) gives exact ownership for a symbolic heap.

Rocq realization

`concrete_heap` in `SP/Semantics.v` is a list of address/field-list cells. Well-formedness is `NoDup (map fst h)`. Rather than define a partial-map union function, `heap_join` in `SP/Semantics.v` is a relation:

```
Definition heap_disjoint (left right : concrete_heap) : Prop :=
  forall address fields fields',
    In (address, fields) left -> In (address, fields') right -> False.

Definition heap_join (left right whole : concrete_heap) : Prop :=
  heap_disjoint left right /\ Permutation (left ++ right) whole.
```

Listing 3.1: Disjoint union is relational and permutation-insensitive.

The permutation makes cell-list order semantically irrelevant. Spatial conjunction is realized by recursion over `spatial_atoms`:

```
Fixpoint spatial_holds_with pred_holds rho atoms h : Prop :=
  match atoms with
  | [] => h = []
  | atom :: rest =>
      exists owned frame,
```

```
        heap_join owned frame h /\
        spatial_atom_holds_with pred_holds rho atom owned /\
        spatial_holds_with pred_holds rho rest frame
  end.
```

Listing 3.2: The fold implementing iterated separating conjunction.

This code has exactly the theoretical interpretation given above: `owned` is the heaplet for the head atom, and `frame` is the disjoint remainder. `points_to_holds` in `SP/Semantics.v` additionally demands a non-null address and equality with a singleton concrete heap. The checked lemmas `points_to_owns_exactly_one` in `SP/Semantics.v` and `points_to_never_owns_nil` in `SP/Semantics.v` expose those two consequences.

Soundness and limitations

Only the top-level `world` stores a proof that its heap has no duplicate addresses. Heaplets are related to that world heap through `heap_join`; they are not separately packaged with well-formedness proofs. This keeps the semantic definitions light, but users proving stronger heap algebra will need lemmas transferring well-formedness through joins.

The field list belongs to one allocated address in this model. It is not the address-arithmetic presentation in which a multi-field record occupies several consecutive heap addresses. Exact ownership and non-null allocation are the facts relevant to the current rules.

3.2 Finite approximants of inductive predicates

Mathematical intention

Let Φ_D be the monotone operator induced by a finite clause environment D. Instead of postulating its least fixed point, define positive-height approximants $P^{(r)}$:

$$P^{(0)}(\bar{a}, h) \equiv \bot,$$

and

$$P^{(n+1)}(\bar{a}, h) \equiv \exists (B \Rightarrow P(\bar{x})) \in D(P),\ \sigma, \rho.\ [\![\sigma\bar{x}]\!]_\rho = \bar{a}\ \wedge\ \rho, h \models B[\sigma] \text{ when every child } Q(\bar{t}) \text{ uses some } r \leq n. \tag{3.2}$$

Thus a predicate holds exactly when it has a finite unfolding derivation of some positive height. Siblings need not have the same height; the parent merely supplies a common upper bound.

Rocq realization

Equation (3.2) is a direct reading of `predicate_holds_rank` in `SP/Semantics.v`:

```
Fixpoint predicate_holds_rank ds rank name arguments h : Prop :=
  match rank with
  | 0 => False
  | S smaller =>
      exists c s rho,
        In c (lookup_definition name ds) /\
        eval_terms rho (map (subst_term s) c.(clause_head)) = arguments /\
        symheap_holds_with
```

```
          (fun p owned =>
             exists child_rank,
               child_rank <= smaller /\
               predicate_holds_rank ds child_rank p.(predicate_name)
                 (eval_terms rho p.(predicate_arguments)) owned)
          rho h (subst_heap s c.(clause_body))
  end.
```

Listing 3.3: Finite ranked predicate membership.

The `rho` inside an unfolding interprets clause variables. The explicit syntactic substitution `s` is the same kind of object later constructed by rule generation. A child rank may be zero syntactically, but membership at rank zero is false, so successful recursive children necessarily have positive rank.

A tagged occurrence does not change membership. It adds the obligation

$$\mathsf{tagBound}(\tau, t, r) = \begin{cases} \top, & t = \mathsf{Anonymous}, \\ r \le \tau(t), & t \text{ named}. \end{cases}$$

This is `tag_bounds_rank` in `SP/Semantics.v`. The wrapper `formula_predicate_holds` in `SP/Semantics.v` existentially chooses r, proves ranked membership, and proves its tag bound.

Soundness and limitations

The semantic predicate is a union of finite approximants, so all successful membership evidence is finite. There is no coinductive interpretation and no infinite heap. The definition also permits an arbitrary syntactic substitution and valuation as witnesses; rule generation is far more restrictive than the semantics. That asymmetry is normal for a sound but incomplete prover.

Ranks are natural numbers. The code calls constraints "ordinal" constraints, but this revision does not represent transfinite ordinals. The terminology records their proof-theoretic role; their concrete carrier is $\mathbb{N}$.

3.3 Disjunction, existential witnesses, tags, and validity

Mathematical intention

A formula chooses its existential witnesses once and shares them between its constraints and its selected disjunct. With free valuations ρ_0, τ_0, write $\rho_0 \oplus \rho_\exists$ and $\tau_0 \oplus \tau_\exists$ for kind-directed merging. Then

$$\begin{aligned} \rho_0, \tau_0, h \models (C; H_1 \vee \cdots \vee H_k) \quad \Longleftrightarrow \quad \exists \rho_\exists, \tau_\exists, i.\ & 1 \le i \le k \wedge \\ & C(\tau_0 \oplus \tau_\exists) \wedge \\ & (\rho_0 \oplus \rho_\exists, \tau_0 \oplus \tau_\exists, h) \models H_i. \end{aligned}$$

Strict and non-strict constraints mean ordinary $<$ and $\le$ between natural-valued tag assignments. A sequent $L \vdash R$ is valid when, in every well-formed world, satisfaction of L implies satisfaction of R.

Rocq realization

The three existential choices in `formula_holds` in `SP/Semantics.v` are `witness_rho`, `witness_tau`, and `sh`. Membership `In sh formula_heaps` implements disjunction. Both merged valuations are then used for the constraints and heap:

```
Definition formula_holds ds base_rho base_tau h f : Prop :=
  exists witness_rho witness_tau sh,
    In sh f.(formula_heaps) /\
    let rho := merge_valuation base_rho witness_rho in
    let tau := merge_tag_valuation base_tau witness_tau in
    constraints_hold tau f.(formula_constraints) /\
    symheap_holds_with (formula_predicate_holds ds rho tau) rho h sh.
```

Listing 3.4: Formula satisfaction chooses shared witnesses.

`merge_valuation` selects the base value for a free variable and the witness value for an existential variable. `merge_tag_valuation` maps an anonymous tag to zero, a free tag through the world assignment, and an existential tag through the witness assignment.

A `world` contains a variable valuation, a free-tag valuation, a concrete heap, and its `heap_wf` proof. `denote` in `SP/Semantics.v` interprets a judgment in that world. The module `SeparationLogicTheory` supplies this world, judgment, denotation, visible-tag enumeration, and tag rank to the generic functor `Framework.Make`; its result is locally named `Cyclic`.

Soundness and limitations

Formula existential witnesses are not computational search witnesses. They live in `Prop` and express semantics. By contrast, graph trace witnesses in Chapter 5 are finite executable data. Confusing these two uses of "witness" obscures the reflection boundary.

Constraint well-formedness can be checked by `formula_wfb` in `SP/Syntax.v`, but `formula_holds` itself interprets every stored constraint. Likewise, a malformed or unhelpful formula is not assigned a magical default meaning by search; it usually just fails to match a generator.

Chapter 4

Definition libraries and benchmark judgments

Mathematical intention

Inductive definitions are data supplied to the semantic fixed-point approximants. For example, the list-segment predicate used in the central worked example is

$$\mathsf{ls}(u, v) \;\equiv\; (u = v \wedge emp) \;\vee\; \exists e.\, u \neq v \wedge u \mapsto e * \mathsf{ls}(e, v). \tag{4.1}$$

An entailment benchmark is not just a pair of formulas: it carries the exact definition environment relative to which both sides are interpreted.

Rocq realization

`SP/Definitions.v` introduces notation-like constructors `F`, `E`, `EQ`, `NEQ`, `PTO`, `PR`, `SH`, and `CL`; they build ordinary values from `SP/Syntax.v`. There is no parser or trusted external AST conversion. The two clauses of Equation (4.1) are reified as follows:

```
Definition ls_clauses : list clause :=
  [CL (SH [EQ (F 0) (F 1)] [] []) [F 0; F 1];
   CL (SH [NEQ (F 0) (F 1)] [PTO (F 0) [E 0]]
     [PR "ls" [E 0; F 1]]) [F 0; F 1]].
```

Listing 4.1: The exact reification of the list-segment clauses.

The main `sl_definitions` in `SP/Definitions.v` table contains 38 named predicate entries, in the source order recorded by the file. They cover list, doubly linked list, tree, path, queue, and auxiliary examples. Separate `atva_definitions` and `songbird_definitions` tables contain only the dependency closures needed by the selected external examples.

`SP/Benchmarks.v` reifies nine judgments: six `benchmark_base_*` inputs, one ATVA input, and two Songbird inputs. The list `selected_benchmarks` in `SP/Benchmarks.v` is a data catalogue; membership in that list is not a claim that the current bounded search solves the judgment. Only explicit theorems or successful computations justify such a claim.

Soundness and limitations

The source comment "reified without a parser" is important for the trust story: these terms are inspected by Rocq's kernel like any other definitions. It also means this revision offers no front-end correctness theorem connecting textual benchmark syntax to reified values.

The large definition library exceeds the implemented rule coverage. It is useful semantic data and a regression corpus, not evidence of completeness. Chapter 10 explains the one named benchmark for which this revision exports an end-to-end validity theorem.

Chapter 5

The generic cyclic framework

5.1 Local rules as validity and countermodel propagation

Mathematical intention

Let judgments denote predicates over worlds. A judgment J is valid when $\forall w.\ w \models J$. A rule with conclusion J and premises $J_0, \ldots, J_{m-1}$ needs the ordinary soundness property

$$(\forall k < m.\ \mathsf{Valid}(J_k)) \Longrightarrow \mathsf{Valid}(J). \tag{5.1}$$

For a cyclic soundness proof, the contrapositive needs more information. From a countermodel $w \not\models J$, the rule must select a premise J_k, a countermodel $w' \not\models J_k$, and a trace relation respected between (J, w) and (J_k, w').

An edge relation contains non-increasing pairs and progressing pairs. For a source tag a, target tag b, and edge e, the semantic obligations are

$$(a, b) \in V_e \Longrightarrow \mathsf{rank}(J_k, w', b) \leq \mathsf{rank}(J, w, a), \tag{5.2}$$

$$(a, b) \in P_e \Longrightarrow \mathsf{rank}(J_k, w', b) < \mathsf{rank}(J, w, a), \tag{5.3}$$

where well-formed edges require $P_e \subseteq V_e$. Equations (5.2)–(5.3) are the semantic bridge between a syntactic trace pair and descent in the object logic.

Rocq realization

`Cyclic/Framework.v` begins with module type `THEORY`. An instantiation supplies `World`, reified `Judgment`, exact and backlink comparisons, `denote`, visible `tags`, and a natural-valued `rank`. The functor defines `Valid j := forall w, holds j w`.

The edge data and its meaning are explicit:

```
Record edge_info : Type := {
  edge_valid : list tag_pair;
  edge_progress : list tag_pair
}.

Definition respects_edge source target w w' e : Prop :=
  (forall a b, In (a,b) e.(edge_valid) ->
     T.rank target w' b <= T.rank source w a) /\
  (forall a b, In (a,b) e.(edge_progress) ->
     T.rank target w' b < T.rank source w a).
```

Listing 5.1: Local trace information and its semantic contract.

The record `rule_instance` in `Cyclic/Framework.v` contains both `rule_theorem`, corresponding to Equation (5.1), and `rule_counter_step`, which returns the premise index, premise record, target world, falsity proof, and `respects_edge` proof. An `axiom_instance` is a conclusion plus a direct proof of its validity.

Soundness and limitations

The generic framework does not prove an arbitrary user-written rule sound by testing it. Constructing `rule_instance` requires proofs of both fields. The executable graph stores those proof-bearing records; erasure during extraction may remove proofs, but the Rocq theorem is checked before extraction.

The framework asks for natural-valued ranks. Object logics needing a different well-founded domain must encode it into this interface or generalize the framework. In the current `SP` instance, the rank of a free tag is simply the world's natural tag valuation.

5.2 Proof graphs and structural well-formedness

Mathematical intention

A regular cyclic derivation is represented by a finite directed graph. Each node is open, closed by an axiom, expanded by a rule to all of its premises, or closed by a backlink. A well-formed graph must satisfy at least:

(i) node zero exists and bears the declared root judgment;

(ii) no node is open;

(iii) rule arity, target bounds, target labels, and edge tag endpoints agree;

(iv) every backlink is non-reflexive, has a unary justifying rule, and links compatible labels;

(v) every stored node is reachable from the root.

These are finite syntactic conditions. They prevent malformed graph data but do not by themselves validate a cycle: a self-sustaining loop without descent would satisfy local rule shape while proving nothing.

Rocq realization

`node_kind` in `Cyclic/Framework.v` has the four constructors `OpenNode`, `AxiomNode`, `RuleNode`, and `BacklinkNode`. A graph is a declared root and a list of nodes addressed by natural indices. The update functions `graph_add_axiom`, `graph_apply_rule`, and `graph_add_backlink` are functional and return `option graph`; none mutates an existing graph.

Applying a rule replaces one matching open node and appends a new open node for each premise. Closing with a backlink checks an existing target, a unary premise, exact agreement between the target and that premise, the theory's `backlink_eqb` between source and target, and unequal node indices. Script execution has a sticky `build_ok` flag: after the first failed command, later commands preserve the failed state.

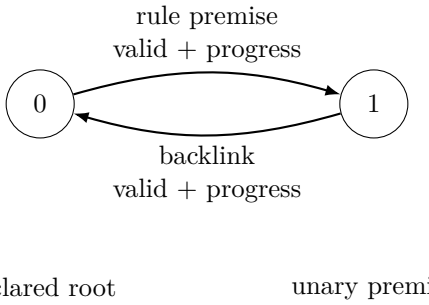


Figure 5.1: The shape of the artificial two-node teaching cycle from `SP/Examples.v`. The strict progress labels, not the circular shape alone, make the cycle acceptable.

The main finite checker is `structural_check` in `Cyclic/Framework.v`. Its `node_wfb` component rejects open nodes, checks proof-package conclusions, checks rule targets and edge information in lockstep, and validates backlink shape. `all_reachableb` expands successors for as many rounds as there are nodes, then verifies membership of every index.

Soundness and limitations

`label_at` falls back to the graph root at an invalid index, but this is not a loophole: every accepted target is separately bounds-checked before the label comparison matters. This illustrates a common executable pattern in the framework: total helper functions are surrounded by acceptance checks carrying the real invariant.

A backlink may compare labels modulo `SP`'s erased tags and constraints, yet its packaged rule conclusion and premise still undergo exact `judgment_eqb` checks. Search currently supplies `exact_backlink_rule goal`, whose premise is exactly `goal`; in practice the search backlink is therefore exact despite the more permissive framework hook.

5.3 Relational closure and the global progress check

Mathematical intention

For trace relations, use the two-element slope algebra $\{\mathsf{Stay}, \mathsf{Decrease}\}$, with decrease dominating composition and union. If a path carries a Stay b followed by b $\mathsf{Decrease}$ c, the composed arc is decreasing. A closed graph path is globally progressing when its composed relation has some decreasing diagonal t $\mathsf{Decrease}$ t. Saturating finite graph paths and tag relations makes this condition executable.

This is a finite-graph presentation of the global trace condition: an infinite path through a finite graph revisits a node; the corresponding loop must carry strict progress on a trace. It is the regular-proof version of infinite descent discussed by Brotherston and Simpson [3].

Rocq realization

`Cyclic/Relations.v` implements the algebra. The function `slope_join` makes `Decrease` dominant, and `put_arc` merges duplicate source/target keys. The function `relation_compose` composes compatible arcs. The function `relation_closure` iterates composition until equality or supplied fuel is exhausted. The analogous `path_closure` composes relations whose graph endpoints meet. Finally:

$$\texttt{relational_trace_check}\mathit{in}\texttt{Cyclic/Relations.v} = \mathsf{forallb}(\mathsf{loop_relation_ok}, \mathsf{path_closure}(\cdots)).$$

A loop is accepted when the closure of its tag relation contains a decreasing diagonal. Non-loop paths impose no check at this final stage.

The framework converts each graph edge into a `path_relation`; valid pairs become arcs and membership in `edge_progress` selects their slope. `relation_fuel` in `Cyclic/Framework.v` is $1 + 3^{w^2}$ for w distinct graph tags, and `graph_path_fuel` is $1 + n^2$ times the relation fuel for n nodes. These finite bounds are then passed to `relational_trace_check` by `relational_check`.

The regression examples in `Cyclic/Relations.v` make the intended algebra concrete: composing stay with decrease produces a decreasing diagonal; a decreasing one-node loop passes; a flat one-node loop fails.

Soundness and limitations

The relational closure check is part of acceptance, but the generic `certificate_sound` in `Cyclic/Framework.v` proof does not consume a theorem reflecting this Boolean closure. Instead, it consumes the explicit finite trace witness described next. Therefore one should describe `relational_check` as an additional conservative acceptance condition, not as the logical premise used by the final infinite-descent proof.

The fuel formulas are deliberately generous finite-state saturation bounds and can be expensive. Their presence does not make graph search complete; graph construction and witness enumeration are independently bounded and incomplete.

5.4 Finite trace witnesses

Mathematical intention

A finite witness chooses, at each node i, either no trace tag yet or one tag t_i, plus an auxiliary potential $p_i \in \mathbb{N}$. On an edge $i \to j$ that leads into a live region, the transition conditions are:

source choice	target choice	obligation
none	none	$p_j < p_i$
none	some tag	true (start following a trace)
some tag	none	false (a selected trace may not disappear)
a	b	$(a, b) \in V_e$ and $((a, b) \in P_e) \vee p_j < p_i$

The auxiliary potential lets the witness traverse finitely many non-progress edges, or delay trace selection, without pretending those steps decrease the semantic rank. A node is *live* if it begins a path of length equal to the number of stored nodes; in a finite graph this is enough to reach a cycle.

Rocq realization

The record `trace_witness` in `Cyclic/Framework.v` stores two total functions: `witness_tag : nat -> option nat` and `witness_potential : nat -> nat`. `option_tag_wfb` checks that a selected tag belongs to a live node's judgment. `trace_transitionb` is the Boolean form of the four-row table, while `trace_transition` is its propositional form. `witness_check` checks all relevant outgoing edges.

The combined Boolean acceptance function is exactly

$$\mathsf{raw_check}(g, tw) = \mathsf{structural_check}(g) \wedge \mathsf{relational_check}(g) \wedge \mathsf{witness_check}(g, tw). \tag{5.4}$$

Soundness and limitations

The witness is global data, not one independent tag choice per path. One choice function must satisfy every checked live edge. This restriction is convenient for a finite soundness certificate and may reject graphs that admit a more path-dependent infinitary trace argument.

Edges into non-live regions impose no witness transition. This is sound for the intended countermodel argument because a false node is later proved live; finite dead regions cannot host the infinite sequence used for contradiction.

5.5 Certificates and lexicographic infinite descent

Mathematical intention

Assume the root is false in some world. At every false graph node, local countermodel propagation chooses a false successor and respects its edge relation. A finite witness then gives a measure that strictly decreases at each chosen step. Such an infinite descent is impossible in a well-founded order, so the root has no countermodel.

Two phases must be combined. Before a trace has been selected, use the auxiliary potential. After selection, use the pair of semantic tag rank and auxiliary potential:

$$M(i,w) = \begin{cases} \mathsf{Before}(p_i), & t_i = \mathsf{None}, \\ \mathsf{Following}(\mathsf{rank}(J_i, w, t_i), p_i), & t_i = \mathsf{Some}(t_i). \end{cases}$$

Encode these as $(1,(p_i,0))$ and $(0,(\mathsf{rank}(J_i,w,t_i),p_i))$, ordered lexicographically. Starting a trace strictly decreases the leading phase bit. A progressing pair decreases the semantic rank. A merely valid pair cannot increase that rank and must decrease the auxiliary potential. Before selection, the auxiliary potential decreases.

Rocq realization

The `certificate` in `Cyclic/Framework.v` record contains the graph and witness, root existence and label evidence, and three semantic obligations:

- `cert_false_live`: every bounded false node is live;
- `cert_advance`: every bounded false node has a bounded false successor whose edge relation is respected;
- `cert_trace_transition`: every edge into a live node follows the finite witness.

These fields are not Booleans. They are dependent proof terms checked by the kernel. `measure_lt_wf` in `Cyclic/Framework.v` obtains well-foundedness from nested strict lexicographic products of $<$ on natural numbers.

`certificate_step_decreases` in `Cyclic/Framework.v` combines `cert_advance`, target liveness, witness transition, and `respects_edge` to produce a false successor with smaller combined measure. The four cases exactly match the preceding transition table.

Finally, `certificate_sound` in `Cyclic/Framework.v` assumes a false root, uses classical double-negation elimination, and performs well-founded induction on the combined measure. The step lemma always supplies a smaller false state, contradicting the induction hypothesis. Hence

$$\mathsf{check_certificate}(c) = \mathsf{true} \quad \Longrightarrow \quad \mathsf{Valid}(\mathsf{root}(c)).$$

Soundness and limitations

There is a subtle but crucial implementation fact: the proof of `certificate_sound` introduces the Boolean-check hypothesis with an anonymous pattern and does not use it. Soundness resides in the semantic fields already stored in `certificate`. The check is valuable as an acceptance interface and is essential to the automated constructor developed in `SP/Certification.v`; it is not an oracle that makes an arbitrary certificate sound.

This separation is strong proof engineering. A manually constructed certificate must provide the hard semantic fields directly. A computed graph obtains them only through proved reflection lemmas. In neither case can a Boolean value inhabit those dependent fields without a theorem.

Chapter 6

The implemented separation-logic rules

This chapter states each mathematical rule before its Rocq package. Empty edge information is used whenever the rule makes no trace claim.

6.1 Identity and tag erasure

Mathematical intention

Exact identity is the axiom

$$\frac{}{F \vdash F}\ \text{id}$$

for a fixed definition environment. Tag erasure is the one-way axiom

$$\frac{}{F \vdash \lfloor F \rfloor}\ \text{erase}$$

where $\lfloor F \rfloor$ removes all ordinal constraints and changes every predicate tag to anonymous. This direction is sound because a named tag adds an upper-bound obligation, while an anonymous tag imposes none. The reverse direction is not generally valid.

Rocq realization

`identity_valid` in `SP/Rules.v` proves implication by assumption, and `identity_axiom` packages it. The tag-erasure proof proceeds from predicate occurrences outward: `erased_predicate_holds` retains the ranked membership witness and discards its bound; `spatial_erase_tags` lifts this over separating conjunction; `heap_erase_tags_holds` lifts it to symbolic heaps; and `tag_erasure_valid` selects the erased image of the original disjunct. `tag_erasure_axiom` packages the theorem.

Soundness and limitations

Exact identity uses structural formula equality, not semantic normalization. The erasure axiom requires the succedent to be exactly the computed erasure of the antecedent. It is not a general "ignore tags whenever convenient" rule, and it never adds a missing constraint or tag bound.

6.2 Ex falso from an explicit contradiction

Mathematical intention

For any right formula R, an inconsistent left formula proves the sequent:

$$\frac{\mathsf{inconsistent}(L)}{L \vdash R}\ \bot L.$$

The implemented decision procedure recognizes only a direct reflexive disequality $t \neq t$ in every disjunct of L. This suffices because the pure conjunction of the selected disjunct would contain a false atom.

Rocq realization

`heap_inconsistentb` in `SP/Syntax.v` searches for an atom on which `pure_reflexive_contradiction` succeeds. Formula inconsistency is `forallb` over disjuncts, including the vacuous truth for an empty disjunction. `heap_inconsistentb_sound` in `SP/Rules.v` reflects the recognized atom into semantic contradiction; `formula_inconsistentb_sound` lifts the argument through the selected disjunct; `ex_falso_valid` and `ex_falso_axiom` close the sequent.

Soundness and limitations

The test does not discover contradictions such as $x = y \wedge x \neq y$, alias conflicts between two points-to atoms, impossible null allocation, or inconsistent tag constraints. Those may be semantically false, but this axiom generator returns `None` unless the narrow check succeeds. Again, this is incompleteness rather than unsoundness.

6.3 Right introduction of an inductive predicate

Mathematical intention

For a clause $c = (B \Rightarrow P(\bar{x}))$ and a substitution σ, the implemented introduction principle is

$$\frac{c \in D(P)}{\sigma B \vdash P(\sigma \bar{x})}\ P\mathrm{R}_c.$$

It has no proof-graph premises because the entire instantiated body appears in the antecedent. Semantically, any finite set of predicate occurrences in σB has a common maximum rank n; the chosen clause then establishes the head predicate at rank $n + 1$.

Rocq realization

`spatial_formula_has_bound` in `SP/Rules.v` inducts through the spatial atoms and combines child ranks with `Nat.max`. The heap-level version is `heap_formula_has_bound`. In `predicate_intro_valid`, satisfaction of the singleton antecedent produces such a bound. The proof constructs an instance of `predicate_holds_rank` at `S bound`. It uses the supplied clause, substitution, and merged valuation, then packages the whole world heap as the single predicate's owned heap with an empty frame.

`predicate_intro_axiom` stores the resulting theorem. It is called an axiom instance because it closes a proof-graph leaf, not because the logical fact is assumed: the field contains the completed Rocq proof.

Soundness and limitations

The rule accepts an untagged singleton predicate on the right and an exactly matching singleton symbolic heap on the left. It is not a general right unfolding inside an arbitrary spatial context. Search also lacks a companion left-unfolding rule at this revision.

The common-rank construction ignores named tag bounds in the antecedent after extracting membership: the resulting right predicate is anonymous, so no new upper-bound obligation is required. A tagged conclusion would need a stronger rule.

6.4 Left disjunction and right choice

Mathematical intention

Disjunction in the antecedent is eliminated by proving both cases:

$$\frac{H \vdash R \quad T \vdash R}{(H \vee T) \vdash R} \vee L.$$

Here T denotes the disjunction of the remaining symbolic heaps. A right disjunction is introduced by choosing a single branch:

$$\frac{L \vdash H}{L \vdash (H \vee T)} \vee R_1.$$

The second rule is sound but asymmetric; another branch might be provable even when the first is not.

Rocq realization

`left_disjunction_rule` in `SP/Rules.v` has two premises built by `left_head_judgment` and `left_tail_judgment`. Its forward theorem case-analyzes the selected antecedent disjunct. Its countermodel theorem uses classical informative excluded middle to decide whether the head-only formula holds in the countermodel world; it selects the corresponding false premise.

`right_disjunction_rule` in `SP/Rules.v` has one premise, the head-only succedent. Its forward proof embeds a witness for the head into the larger disjunction. Its countermodel proof observes that if the larger succedent is false, the chosen head succedent is false as well. Both rules use empty trace edges.

Soundness and limitations

The left rule preserves the original formula constraints on both branches. Existential witnesses remain local to satisfaction of each premise formula; no computational witness is transferred between branches.

The right generator only chooses the head disjunct. It neither produces one candidate per right disjunct nor backtracks over alternative choices. This is one of the most visible sources of search incompleteness.

6.5 Exact backlinks and the teaching-only progressing loop

Mathematical intention

An exact backlink is semantically an identity step:

$$\frac{J}{J}\ \text{backlink}.$$

For each visible tag t, the edge admits (t, t) as non-increasing and claims no progress. Such an edge can participate in a valid cyclic proof only if progress is supplied elsewhere on every relevant loop.

Rocq realization

`identity_edge` in `SP/Rules.v` maps visible judgment tags to identity pairs. `identity_edge_respected` proves equality of source and target ranks, and `exact_backlink_rule` packages the unary identity theorem plus its countermodel step.

The same file also defines `progressing_valid_rule`. Given an independent proof $\mathsf{Valid}(J)$, it constructs a unary J/J rule whose edge declares (t, t) progressing. Its countermodel field is discharged ex falso because a valid conclusion has no countermodel. `SP/Examples.v` uses this rule to exercise the cyclic checker.

Soundness and limitations

`progressing_valid_rule` is not a search rule for discovering inductive progress. Its premise is already accompanied by a proof that the conclusion is valid. Therefore the two-node example built from it demonstrates graph and trace plumbing, not a circular proof of a previously unknown separation-logic entailment.

An exact backlink edge makes no strict progress. Since the current search generates no progressing object-logic rules, backlinks cannot by themselves produce an accepted genuine cyclic search result. They remain useful infrastructure for future rules.

Chapter 7

Executable rule generation

7.1 One-way ordered matching

Mathematical intention

To instantiate Rule 6.3, search must find a clause whose head and body match the goal. The intended computation is matching, not symmetric unification: variables in the clause pattern may be assigned target terms, but target variables are treated as syntax. Lists match position by position and must have equal length. The process is deterministic for one clause:

$$\mathsf{head}(c) \triangleright \mathsf{targetArgs} \quad \text{then} \quad \mathsf{body}(c) \triangleright \mathsf{targetBody}.$$

The substitution accumulated by the head match constrains the body match. Pure atoms must have the same constructor, points-to atoms retain their list position, and predicates retain both name and list position.

Rocq realization

`SP/RuleGeneration.v` implements separate matchers for two terms, pure atoms, pure lists, points-to atoms, points-to lists, predicates, predicate lists, and entire heaps. The heap order is pure atoms, points-to atoms, then predicate atoms. The essential shape is:

```
Definition unify_heap pattern target s : option substitution :=
  match unify_pures pattern.(heap_pure) target.(heap_pure) s with
  | Some s1 =>
      match unify_points_list pattern.(heap_points)
                              target.(heap_points) s1 with
      | Some s2 =>
          unify_predicate_list pattern.(heap_predicates)
                               target.(heap_predicates) s2
      | None => None
      end
  | None => None
  end.
```

Listing 7.1: Ordered heap matching; no permutation search occurs.

`predicate_intro_for_clause` in `SP/RuleGeneration.v` first obtains the membership proof that the clause occurs in the named definition. It matches the head, then the body, constructs

`predicate_intro_axiom`, and finally checks that this theorem package's complete conclusion is exactly the original goal. The final check is a defense against a matcher that produced a substitution insufficient to reconstruct the intended judgment.

`first_predicate_intro` in `SP/RuleGeneration.v` tries clauses in definition order and returns the first candidate. `predicate_intro_candidate` is defined only for singleton left and right disjunctions, where the right heap is exactly one predicate and has no pure or points-to atoms.

Soundness and limitations

The matcher does not search permutations of conjunction lists, perform associative-commutative matching, or solve pure equalities. This is stricter than the heap semantics, where cell-list order is erased by permutation in `heap_join`. A mathematically identical reordering may therefore prevent generation.

Pattern `Var x` can bind either a free or existential variable. The utility `existential_substitutionb` in `SP/Automation.v` recognizes substitutions that bind only existential variables, but the predicate introduction generator does not use that filter. Soundness does not depend on the filter: `predicate_intro_valid` is quantified over every executable substitution, and the final conclusion equality must hold. The utility records a useful side condition for future transformations.

7.2 Candidate priority and conservative stubs

Mathematical intention

Rule generation should prefer leaf closure over branching. If more than one sound axiom applies, any is acceptable; order only affects search behavior. Ordinary rule candidates may form a list, whose order becomes depth-first search order.

Rocq realization

`generate_axiom` in `SP/RuleGeneration.v` uses left-biased `first_some` in this exact priority:

1. exact identity;
2. explicit pure inconsistency;
3. tag erasure;
4. predicate introduction.

`generate_rules` in `SP/RuleGeneration.v` returns the optional left-disjunction candidate followed by the optional right-disjunction candidate. A disjunction candidate is created only when at least two symbolic heaps are present.

Seven named generator boundaries currently return `None` unconditionally:

- `generate_equality_substitution`;
- `generate_constraint_simplification`;
- `generate_bound_introduction`;
- `generate_points_to_instantiation`;
- `generate_left_unfolding`;
- `generate_right_unfolding`; and
- `generate_partial_backlink`.

Soundness and limitations

These stubs are a positive part of the trust story. An unsupported operation does not manufacture a graph node and hope a later checker will reject it; it has no candidate. Consequently the implemented prover is much smaller than a full cyclic separation-logic calculus.

The priority list is operational, not logical. For example, if identity succeeds, a later predicate-introduction interpretation of the same goal is never attempted. This cannot harm soundness, but it can affect proof shape and future performance extensions.

Chapter 8

Reflection: from a checked graph to semantic evidence

8.1 Why reflection is needed

Mathematical intention

A Boolean equation such as $\mathsf{rawCheck}(g, t) = \mathsf{true}$ is easy to compute but does not by itself provide the dependent functions required by the generic certificate. A reflection layer must prove, once and for all, that the successful finite checks reveal:

1. a real root and its exact label;
2. a false successor and countermodel from every false bounded node;
3. liveness of every false bounded node;
4. a propositional trace transition on every relevant edge.

The difficult item is countermodel propagation. The graph checker knows only node shape. The rule package stored at the node knows how to turn a countermodel of its conclusion into a countermodel of one premise. Reflection must align that selected premise with the checked target index and label.

Rocq realization

`SP/Certification.v` builds this bridge in layers. The early lemmas reflect membership and index-based checks:

- `pair_mem_spec` turns a true pair lookup into `In`;
- `all_nodes_wfb_from_nth` and `structural_node_wfb` recover the well-formedness equation for a particular node;
- `structural_root_bound` and `structural_root_label` recover root evidence;
- `targets_wfb_nth` aligns a premise selected by `rule_counter_step` with its graph target, bounds proof, and label;
- `node_edge_from_rule_nth` recovers the corresponding proposition `edge_at`.

The main lemma `structurally_checked_advance` in `SP/Certification.v` analyzes the payload of a false node. An open node contradicts structural well-formedness. An axiom node contradicts its stored validity theorem. At a rule node, `rule_counter_step` chooses a false premise and semantic world, while `targets_wfb_nth` locates its checked graph target. At a backlink node, the unary target checks provide the same alignment directly.

There is a Rocq-specific dependent-elimination point worth noticing. The proof destructs the executable `node_at g i` lookup rather than first proving $\exists n$ in `Prop`. The desired result contains dependent witnesses in `Type`; eliminating an arbitrary existential proof from `Prop` into such data would be disallowed. Computing the lookup preserves the needed information in the eliminable sort.

Soundness and limitations

Structural checking alone gives a false successor, but it does not give an infinite path for arbitrary nodes. The lemma `false_has_path_of_length` in `SP/Certification.v` iterates the advance theorem for any finite fuel while maintaining falsity. Instantiating the length with the graph size proves `structurally_checked_false_live`. This is why the certificate's liveness field is restricted to false nodes.

The bridge uses the semantic countermodel theorem stored in every rule package. If a future generator introduces a rule without a correctly proved `rule_counter_step`, it will fail at construction of the rule instance, not merely in this reflection proof.

8.2 Reflecting the finite witness

Mathematical intention

The Boolean trace transition must imply its propositional four-case counterpart. That counterpart is consumed by the descent lemma `certificate_step_decreases`. Whole-graph witness success must then specialize to an arbitrary actual edge into a live target.

Rocq realization

`trace_transitionb_sound` in `SP/Certification.v` performs the four-way case analysis on source and target tag choices. Boolean pair membership is reflected through `pair_mem_spec`; natural less-than is reflected through `Nat.ltb_lt`. `witness_edges_from_nth` selects the checks for a node index, and `witness_check_transition` selects the particular outgoing edge and rewrites the target liveness test to true.

With these pieces, `certify_raw_check` in `SP/Certification.v` constructs the generic certificate:

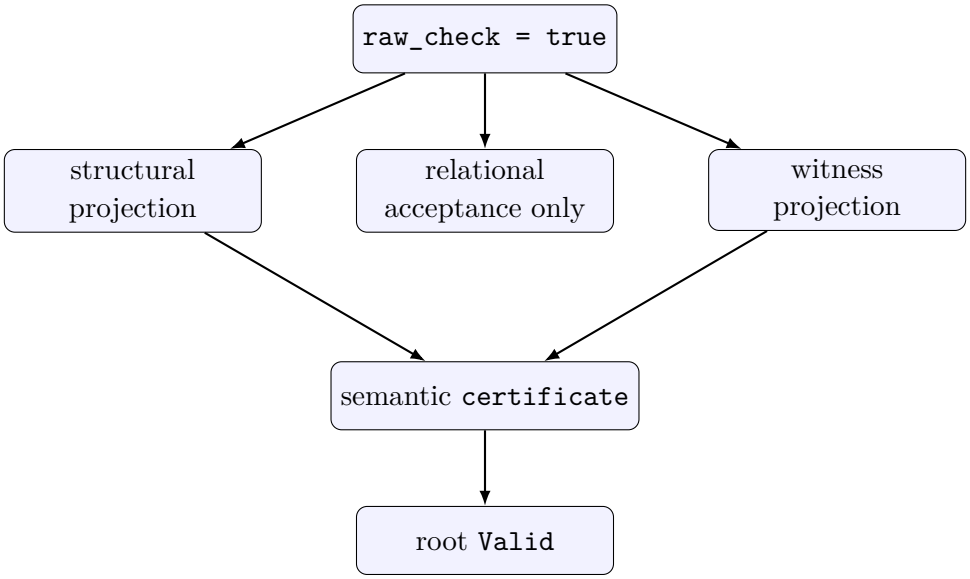
```
Definition certify_raw_check g tw
    (Hcheck : Cyclic.raw_check g tw = true) : Cyclic.certificate.
Proof.
  refine {| Cyclic.cert_graph := g;
            Cyclic.cert_witness := tw;
            Cyclic.cert_root_bound := ...;
            Cyclic.cert_root_label := ...;
            Cyclic.cert_false_live := _;
            Cyclic.cert_advance := _;
            Cyclic.cert_trace_transition := _ |}.
  (* Fields are discharged by structural and witness reflection lemmas. *)
Defined.
```

Listing 8.1: The semantic certificate is constructed from proved projections.

The public theorem `raw_check_sound` in `SP/Certification.v` constructs this certificate and applies generic `certificate_sound`.

Soundness and limitations

Equation (5.4) contains three conjuncts. The certificate constructor explicitly projects `structural_check` and `witness_check`; it does not project or reflect `relational_check`. Nevertheless, callers must supply the full `raw_check = true` equation, so the relational condition is an additional acceptance gate. The precise relationship is:



This is stricter than the semantic constructor requires, not weaker. Removing the relational check would change which graphs the executable prover accepts, even though the existing witness-based soundness proof might still apply. Such a change would therefore be an interface and policy change, not a trivial proof cleanup.

8.3 Acyclic leaves use the same certificate interface

Mathematical intention

A one-node graph closed by a valid axiom has no outgoing edge and no infinite path. Its structural check passes, and both progress checks are vacuous. It should still be usable through the same generic certificate API as a cyclic graph.

Rocq realization

`axiom_graph` in `SP/Certification.v` creates the one-node graph. `empty_trace_witness` selects no tag and potential zero everywhere. `axiom_graph_raw_check` proves the full Boolean check by reduction; `certify_axiom` passes that equation to `certify_raw_check`. Search does not need a special semantic soundness theorem for leaf-only success.

Soundness and limitations

The word “certificate” does not imply that the graph is cyclic. It names the uniform object consumed by `certificate_sound`. This matters in Chapter 10: the benchmark proof is a certified one-node axiom graph, despite residing in a cyclic-prover architecture.

Chapter 9

Graph exploration and the public search API

9.1 First-open-node depth-first search

Mathematical intention

At each graph state, select a deterministic open node, enumerate successors in a fixed priority order, and recursively explore each successor before moving to the next. A closed graph is a candidate result only after all acceptance checks and a finite-witness search succeed. Fuel bounds the number of expansion levels on one attempt.

Rocq realization

`first_open_from` in `SP/GraphSearch.v` scans the node list from index zero and returns the first `OpenNode`. Since rule application appends premises, this is the first open node in list order, not a heuristic best goal.

For that node, `expand_graph` in `SP/GraphSearch.v` concatenates successors in this exact order:

1. the single candidate from `generate_axiom`, if any;
2. exact backlinks to prior indices in ascending order $0, \ldots, i-1$;
3. ordinary generated rules, left-disjunction before right-disjunction.

`bounded_graph_search_from` in `SP/GraphSearch.v` first tries to check the current graph. If it is not accepted and fuel is positive, it recursively explores each expanded graph with predecessor fuel. This is a conventional list-ordered depth-first search.

Soundness and limitations

Fuel is decreased on every graph expansion, not per completed proof branch. A rule adding two premises consumes one level immediately; closing those premises may consume further levels. Search order can therefore determine whether a proof is found at a fixed depth.

Backlinks are considered before ordinary rules, but a non-progressing closed cycle fails checking and search continues with later successors. Because no generated separation-logic rule currently has a progressing edge, genuine cyclic success is not expected from this rule set.

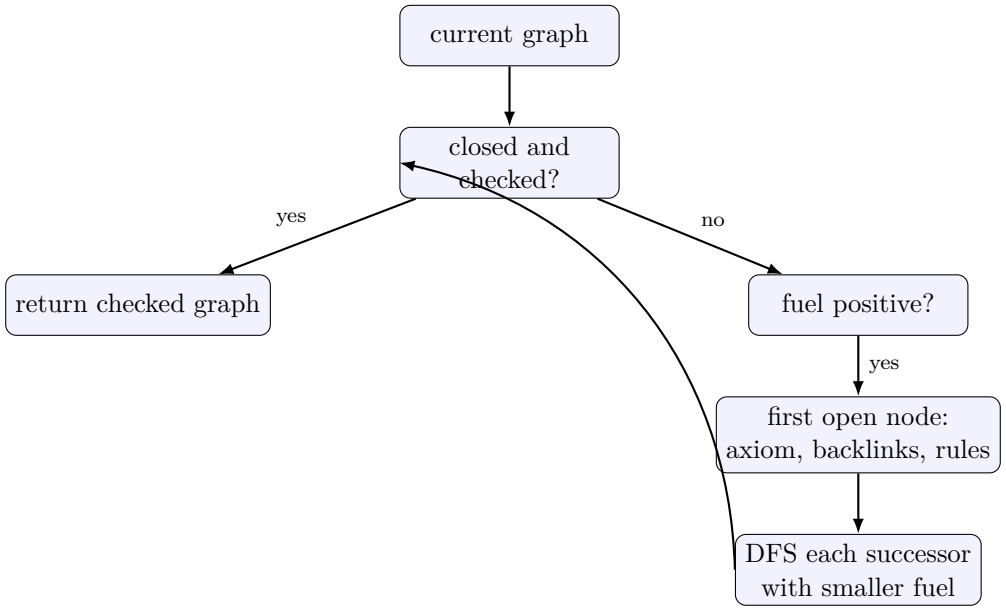


Figure 9.1: Operational control flow of `bounded_graph_search_from`.

9.2 Witness enumeration and closed-graph checking

Mathematical intention

Given a syntactically closed graph, first try cheap witness guesses. If those fail, enumerate every node-local choice from the finite domain consisting of "no tag" plus its visible tags, and every potential in a fixed finite range. Return the first witness that makes the combined checker true.

Rocq realization

`witness_candidates` in `SP/GraphSearch.v` begins with:

1. an everywhere-empty tag witness with zero potentials;
2. a heuristic selecting the first visible tag at every live node, again with zero potentials;
3. the Cartesian enumeration of tag assignments and potentials.

For a graph of n nodes, each potential ranges over `List.seq 0 (S n)`, namely $0, \ldots, n$. `choose_lists` forms Cartesian products. The enumeration is complete only for these finite choice sets, which are sufficient for the intended node-potential witnesses but can be combinatorially large.

`checked_graph` in `SP/GraphSearch.v` stores g, tw, and the equation $\mathsf{raw_check}(g, tw) = \mathsf{true}$. The constructor `check_graph_with_witness` is defined by destructing the Boolean checker, so the success branch receives the required equation. `check_closed_graph` also demands that no open node remains. `certify_searched_graph` converts the result through `certify_raw_check`.

Soundness and limitations

The witness enumeration is an acceptance search nested inside proof-graph search. Its cost grows as a product of per-node tag choices and $(n+1)^n$ potential assignments. The empty and heuristic witnesses are important fast paths, especially for acyclic graphs where liveness is false and witness conditions are vacuous.

9.3 Iterative deepening and the meaning of `solvedb`

Mathematical intention

Iterative deepening repeats bounded DFS at depths $d_{\min}, d_{\min}+1, \ldots, d_{\max}$. A public result should contain both a certificate and evidence that its root is the normalized input goal.

Rocq realization

`SP/Search.v` defines `search_config` with `min_depth` and `max_depth`; the default is 1 through 8. A `verified_proof goal` contains a generic certificate, an equality between its root and `normalize_judgment goal`, and a successful certificate-check equation. `verified_proof_valid` in `SP/Search.v` applies generic soundness and rewrites the root equality. Since normalization is currently identity, the final theorem is exactly about `goal`.

`generated_graph_strategy` runs bounded graph search, reflects the raw check into a certificate, and uses decidable judgment equality to validate the root. `search_depths` tries increasing depths; `search` computes the number of attempts from the inclusive interval. Finally:

```
Definition solvedb config goal : bool :=
  match search goal config with Some _ => true | None => false end.

Theorem search_sound config goal :
  solvedb config goal = true -> Cyclic.Valid goal.
```

Listing 9.1: The Boolean public query and its one-way theorem.

Soundness and limitations

There is no converse theorem. The correct readings are:

$$\begin{aligned} \mathsf{solvedb}(c, J) = \mathsf{true} &\quad\Rightarrow\quad J \text{ is semantically valid,} \\ \mathsf{solvedb}(c, J) = \mathsf{false} &\quad\Rightarrow\quad \text{no accepted proof was found within this bounded strategy.} \end{aligned}$$

False does not imply invalidity, satisfiability of the antecedent, existence of a countermodel, or failure at larger depth. This limitation applies even when the configured interval has been completely searched.

Chapter 10

Complete vertical slice: `benchmark_base_19_1`

This chapter follows one judgment through every layer. It is the most compact way to see how a mathematical entailment becomes `benchmark_base_19_search_valid` in `SP/Search.v`.

10.1 The mathematical entailment

Mathematical intention

The benchmark is

$$x \neq z \ \wedge \ x \mapsto y * \mathsf{ls}(y, z) \quad \vdash \quad \mathsf{ls}(x, z). \tag{10.1}$$

Compare its antecedent with the recursive clause of Equation (4.1):

$$u \neq v \ \wedge \ u \mapsto e * \mathsf{ls}(e, v) \quad \Rightarrow \quad \mathsf{ls}(u, v).$$

The substitution $u \mapsto x$, $v \mapsto z$, $e \mapsto y$ makes the clause body exactly the antecedent and the clause head exactly the succedent. Therefore Rule 6.3 closes the entailment without premises.

Semantically, suppose the left side holds. The recursive child $\mathsf{ls}(y, z)$ has some finite rank r. There are finitely many predicate atoms in the body (here, one), so choose a common bound $n \geq r$. The recursive clause witnesses $\mathsf{ls}(x, z)$ at rank $n + 1$, owning the entire split heap. That is exactly the proof of `predicate_intro_valid` specialized to this clause and substitution.

Rocq realization

The judgment in `SP/Benchmarks.v` is:

```
Definition benchmark_base_19_1 : judgment :=
  mk_judgment sl_definitions
    (mk_sequent
      (singleton_formula
        (SH [NEQ (F 0) (F 2)] [PTO (F 0) [F 1]]
          [PR "ls" [F 1; F 2]]))
      (singleton_formula (SH [] [] [PR "ls" [F 0; F 2]]))).
```

Listing 10.1: Exact source for the worked benchmark.

The path through computation is deterministic:

1. `search_at_depth 1` calls `generated_graph_strategy` on the identity-normalized goal.
2. `bounded_graph_search` starts with one open node at index zero.
3. The open graph is not closed, so `expand_graph` asks for an axiom.
4. Identity fails, inconsistency fails, and tag erasure fails. Predicate introduction recognizes the singleton right predicate `"ls"`.
5. `first_predicate_intro` tests `ls_clauses` in order. The base equality clause does not reconstruct the target body. The recursive clause matches.
6. Ordered head matching binds its formal endpoints to `F 0` and `F 2`. Ordered body matching binds `E 0` to `F 1`; pure, points-to, and predicate lists then agree exactly.
7. `predicate_intro_axiom` is built from `predicate_intro_valid`; the complete conclusion equality check succeeds.
8. `graph_add_axiom` replaces node zero with `AxiomNode`. No new node is added, so the graph has exactly one node.
9. On the recursive call with fuel zero, `check_closed_graph` runs before the fuel case. The graph is closed. The empty trace witness passes: structure is correct, there are no edge paths or loops, and the sole valid axiom node cannot be a live false node.
10. The returned `checked_graph` carries the full raw-check equation. `certify_raw_check` creates the semantic certificate, and root equality produces a `verified_proof benchmark_base_19_1`.

The source records the computation and the final theorem:

```
Example general_clause_introduction_search :
  solvedb {| min_depth := 1; max_depth := 1 |}
    benchmark_base_19_1 = true.
Proof. vm_compute. reflexivity. Qed.

Theorem benchmark_base_19_search_valid : Cyclic.Valid benchmark_base_19_1.
Proof.
  apply (search_sound {| min_depth := 1; max_depth := 1 |}).
  vm_compute. reflexivity.
Qed.
```

Listing 10.2: From a computed Boolean to semantic validity.

Soundness and limitations

The phrase “one-node certified proof” is literal: the graph contains a single axiom node whose validity theorem is predicate introduction. It is not a one-node cyclic loop and it uses no backlink. The cyclic framework still adds value by giving the leaf a uniform checked-graph and semantic-certificate path.

The example demonstrates one exact clause instance. It does not show that arbitrary list-segment entailments, arbitrary reorderings of the body, or the other eight selected benchmarks are solved. In particular, `unsupported_open_goal_is_bounded_non_success` in `SP/Search.v` computes false for `benchmark_base_01_1` at depth zero and carefully names the result “bounded non-success.”

Chapter 11

Examples, tests, and automation utilities

11.1 Accepted and rejected teaching graphs

Mathematical intention

Small graphs should test independent boundaries: an acyclic leaf should pass; a correctly progressing cycle should pass; an open graph, a reflexive backlink, and a flat cycle should fail for different reasons.

Rocq realization

`SP/Examples.v` first packages $emp \vdash emp$ with `identity_axiom`. It turns that leaf into a certificate with `certify_axiom`, then applies `certificate_sound`. This is an ordinary genuine validity theorem and a minimal certificate regression.

The two-node graph in Figure 5.1 uses a tagged identity judgment. Its judgment is proved valid independently by `identity_valid`; that proof is supplied to `progressing_valid_rule`. The graph applies the rule at node zero and backlinks node one to zero using another instance of the same progressing rule. The witness selects tag zero everywhere. Both edges claim strict progress, so `teaching_cyclic_raw_check` reduces to true and the generic certificate proves `teaching_cyclic_valid`.

The rejected cases separate concerns:

- `rejected_open_graph` fails structural checking;
- `rejected_malformed_backlink` stores a self-backlink and fails the non-reflexivity check;
- `rejected_nonprogressing_cycle` uses exact backlink rules on both edges, so the global progress condition fails.

Soundness and limitations

The cyclic example is intentionally artificial. Since validity is an input to the progressing rule, it cannot serve as evidence that search discovered an inductive cyclic proof. The genuine search successes explicitly present in `SP/Search.v` are the general exact-identity example and the clause introduction proof for `benchmark_base_19_1`.

11.2 Regression tests for generators

Mathematical intention

Executable generators need positive and negative unit tests that pin down their accepted shapes without claiming completeness.

Rocq realization

`SP/Tests.v` checks positive generation for identity, explicit inconsistency, tag erasure, benchmark-19 predicate introduction, left disjunction, and right disjunction. It also checks rejection of unmatched predicate introduction and of right splitting for a singleton succedent. Two semantic/syntax units record identity normalization and exact one-cell points-to ownership.

Most simple tests close by reflexivity; larger reified computations use `vm_compute`. These are kernel-checked equations about the actual generators. They are not external test-runner assertions.

Soundness and limitations

The tests characterize selected cases only. Their greatest documentary value is identifying intended generator domains. For example, the positive left split test on `benchmark_base_21_1` shows that a rule candidate can be generated; it does not say the resulting branches are both solved.

11.3 Verified automation utilities

Mathematical intention

Small verified helpers can establish side conditions before a future rule is implemented. A strict self-constraint $t < t$ is inconsistent; substitutions used for clause-local instantiation may be restricted to existential-domain bindings; and fresh identifiers can be chosen above the maximum used id.

Rocq realization

`SP/Automation.v` provides:

- `constraints_inconsistentb` and its semantic soundness lemma for a directly recognized strict self-constraint;
- `existential_substitutionb`, which checks the kind of every binding domain;
- fresh free/existential variable and free-tag constructors using one plus a folded maximum;
- `normalization_candidate` and a semantic equivalence proof, which is reflexive because normalization is identity.

Soundness and limitations

These utilities are not wired into `generate_axiom` or `generate_rules` except indirectly through shared syntax and imports. In particular, inconsistent ordinal constraints do not currently trigger the ex-falso generator. A theorem existing in `Automation.v` does not imply that search calls it.

Chapter 12

Limitations and precise claims

The current implementation is best understood as a small proof-producing core, not as a complete port of a mature separation-logic prover. This positioning matches the historical contrast between a general cyclic proof architecture and a logic-specific automated procedure [2, 4].

Mathematical intention

Soundness and completeness answer different questions. Soundness asks whether every reported proof denotes a valid entailment. Completeness asks whether every valid entailment is eventually reported. The code establishes the first for successful search and makes no general claim about the second.

Rocq realization

Table 12.1 collects the operational restrictions that most often explain non-success.

Table 12.1: Explicit limitations of revision `08f6cd8834a3fc22088a064025abb9ee65c7f545`.

Area	What the implementation does	Consequence
Identity normalization	`normalize_heap`, `normalize_formula`, `normalize_sequent`, and `normalize_judgment` return their input.	Semantic equivalents are not canonicalized; equality and matching remain representation-sensitive.
Ordered matching	Clause head, pure atoms, points-to atoms, predicate atoms, and arguments match in list order, one way from pattern to target.	No commutative matching, atom permutation, equality solving, or alternative substitution search is performed.
Right disjunction	The generator chooses only the first symbolic heap in a multi-disjunct succedent.	A proof requiring another right branch may fail even when that branch is directly provable.

Area	What the implementation does	Consequence
Backlinks	Search proposes `exact_backlink_rule` to earlier nodes; its edge carries identity valid pairs and no progress pairs.	The current generated rules cannot create a genuine progressing search cycle.
Unsupported rules	Equality substitution, constraint simplification, bound introduction, points-to instantiation, left/right unfolding, and partial backlink generators return `None`.	Their names document intended boundaries; they contribute no proof power at this revision.
Inconsistency	Ex falso recognizes only a direct $t \neq t$ in every left disjunct.	Many unsatisfiable antecedents are not closed.
Predicate introduction	The right side must be one untagged predicate in one heap; the left side must be one heap exactly matching an instantiated clause body.	Framed, reordered, tagged, or partially unfolded variants are outside the generator's domain.
Search bounds	Graph DFS is fuel-bounded and iterative deepening stops at the configured maximum. Witnesses are enumerated only after graph closure.	Runtime and result depend on configuration and successor order.
Boolean result	`solvedb = true` has theorem `search_sound`; no converse theorem exists.	`false` means bounded non-success, never semantic invalidity.
Input path	Definitions and benchmarks are handwritten reified values.	There is no parser, parser-correctness theorem, CLI, or OCaml front-end in this tutorial or module set.

Soundness and limitations

The limitations interact. A valid goal might first require normalization, then a non-head right choice, then left unfolding, none of which the current search performs. Increasing depth cannot compensate for a missing successor. Conversely, adding more generators without proved rule packages would violate the central architecture. Extension should increase the set of theorem-backed successors, not weaken the checker.

Chapter 13

Trust boundary and assumptions

Mathematical intention

A proof-producing prover should separate mathematical assumptions, executable decisions, and external data. The strongest useful claim is not "the program seems to work," but "under the explicitly printed logical assumptions, the Rocq kernel accepts a theorem whose statement is semantic validity."

Rocq realization

At the documented revision, rebuilding `SP/Search.v` prints:

```
Axioms:
Description.constructive_definite_description :
forall (A : Type) (P : A -> Prop),
(exists!  x :  A, P x) -> {x :  A | P x}
Classical_Prop.classic :  forall P : Prop, P  ~P
```

for both `search_sound` and `benchmark_base_19_search_valid`. Rebuilding `SP/Examples.v` prints only `Classical_Prop.classic` for `teaching_acyclic_valid` and `teaching_cyclic_valid`.

The sources of these assumptions are visible. Generic `certificate_sound` applies classical double-negation elimination. `SP/Rules.v` uses informative excluded middle while selecting the false branch of the left-disjunction rule; its standard-library construction brings in constructive definite description. These are standard-library axioms, not project-local admitted theorems.

The trusted and checked components can be summarized as follows:

Component	Role in the claim
Rocq kernel and standard library	Check definitions, proof terms, well-founded orders, list/permutation facts, and the explicitly reported classical assumptions.
Reified definitions/benchmarks	Handwritten data. The kernel checks their types and reductions, but there is no external-parser correspondence theorem.
Rule generators	Computational filters that can return only pre-proved `axiom_instance` or `rule_instance` values.
Graph and witness checks	Executable acceptance conditions. Their useful semantic projections are justified in `SP/Certification.v`.

Relational trace check	Additional executable acceptance filter; not projected into the semantic certificate used by the final descent proof.
`vm_compute` equations	Kernel-accepted computations establishing the specific Boolean results shown in examples and the benchmark theorem.
Search strategy	Untrusted for completeness and performance claims, but safe for validity because any returned object must pass the proof-producing path.

Soundness and limitations

There are no project-declared axioms or admitted proof holes in the documented `SP` and required `Cyclic` source modules. This observation does not remove the printed standard-library assumptions. It also does not certify the intended transcription of every external benchmark; it certifies the reified judgments that actually occur in Rocq.

No soundness statement is made about an extracted program after arbitrary modification, a textual input not connected to the reified term, or a proposed graph that has not been converted to a `verified_proof`. The public theorem begins at the actual `solvedb` computation over a Rocq `judgment`.

Chapter 14

A theorem-first extension checklist

The safest way to extend this prover is to work backward from the theorem that the new executable candidate must carry.

Mathematical intention

Suppose a new transformation maps a conclusion J to premises $J_0, \ldots, J_{m-1}$ and edge relations $e_0, \ldots, e_{m-1}$. Before writing a generator, establish:

$$(\forall k < m.\ \mathsf{Valid}(J_k)) \Rightarrow \mathsf{Valid}(J),$$
$$w \not\models J \Rightarrow \exists k, w'.\ w' \not\models J_k \wedge \mathsf{respectsEdge}(J, J_k, w, w', e_k).$$

If the rule claims progress, prove strict rank decrease for exactly those pairs. If it changes tags or constraints, first prove the corresponding semantic monotonicity or witness-transport lemma.

Rocq realization

Use the following checklist in order.

1. **State the mathematical rule.** Specify its conclusion, every premise, and all side conditions without mentioning the search heuristic.
2. **Prove semantic helper lemmas in `SP/Semantics.v` or `SP/Rules.v`.** Typical obligations concern heap splitting, ranked unfolding, substitution, tag bounds, or constraint transport.
3. **Prove the forward theorem.** Match the type `premises_valid premises -> Valid conclusion`. Keep this theorem useful independently of graph search.
4. **Prove countermodel propagation.** Construct the dependent premise index and target world required by `rule_counter_step`. Do not derive it from an unproved Boolean heuristic.
5. **Justify edge information.** Prove `respects_edge` for every selected countermodel step. Declare a progressing pair only when the semantic rank is strictly smaller.
6. **Package a `rule_instance` or `axiom_instance`.** At this point all logical trust is in kernel-checked fields; the generator will only decide applicability.
7. **Implement a conservative generator.** Return `None` whenever a side condition is undecided or false. Reflect each successful Boolean side condition with a theorem before constructing the package.

8. **Check the complete generated conclusion.** As predicate introduction does, compare the package conclusion with the requested goal after matching or normalization.

9. **Insert the candidate deliberately into priority order.** Record whether it branches and how it interacts with axiom, backlink, and existing rule priority.

10. **Add focused positive and negative tests.** Test the generator's boundary, graph installation, raw checking, and at least one semantic theorem through `search_sound`.

11. **Inspect assumptions.** Run `Print Assumptions` on the new public theorem and explain any change from Chapter 13.

12. **Keep failure modest.** Unless a decision procedure and completeness theorem have actually been proved, retain the interpretation of false as bounded non-success.

Soundness and limitations

A new Boolean check alone does not add proof power. A new theorem package that no generator reaches does not add search power. A new rule that reaches a graph but cannot produce a global trace witness does not add accepted cyclic proofs. The checklist intentionally covers all three boundaries.

For a genuine cyclic separation-logic extension, the pivotal task is a rule whose `rule_counter_step` exposes a strictly decreasing ranked predicate trace. Once that local fact is packaged as edge progress, the existing graph, relational, witness, reflection, and lexicographic-descent machinery can be reused.

Appendix A

Mathematics-to-Rocq correspondence

Table A.1: Quick correspondence between the mathematical presentation and source symbols.

Mathematical object	Rocq realization	Important detail
Location and null	`location` in `SP/Semantics.v`, `nil_location`	Both are naturals; null is zero.
Finite partial heap	`concrete_heap`, `heap_wf`	Association list with no duplicate address in a world.
Disjoint union $h_1 \uplus h_2 = h$	`heap_disjoint`, `heap_join`	Join uses permutation of list concatenation.
Exact $x \mapsto \bar{y}$	`points_to_holds`	Non-null address and exactly one cell.
emp and separating conjunction	`spatial_holds_with`	Empty atom list owns `[]`; recursion splits owned heap and frame.
Pure conjunction	`pures_hold`	Recursive conjunction over a list.
Symbolic heap	`symheap`, `symheap_holds_with`	Pure part plus spatial fold of points-to atoms followed by predicates.
Disjunction	`formula_heaps`, membership in `formula_holds`	Empty list denotes false; a selected heap is existential.
Existential witnesses	`merge_valuation`, `merge_tag_valuation`	Selected once per formula satisfaction derivation.
Finite predicate approximant	`predicate_holds_rank`	Rank zero is false; children of rank Sn have some rank at most n.
Tag upper bound	`tag_bounds_rank`	Anonymous is unconstrained; named rank is bounded by its valuation.

Mathematical object	Rocq realization	Important detail
Ordinal constraint	`constraint_holds`	Natural-number $<$ or $\leq$, not transfinite arithmetic.
Entailment validity	`sequent_holds`, `denote`, `Cyclic.Valid`	Universal quantification over well-formed worlds.
Non-increasing/progress pair	`edge_valid`, `edge_progress`, `respects_edge`	Progress pairs must also be valid pairs.
Local inference rule	`rule_instance`	Contains both forward validity and countermodel propagation.
Regular derivation	`graph`, `node_kind`	Finite index-addressed graph with explicit backlinks.
Global trace condition	`relational_check`, `witness_check`	Both are required by `raw_check`.
Trace start/continuation	`trace_witness`, `trace_transition`	Optional selected tag plus auxiliary natural potential.
Infinite descent measure	`descent_measure`, `measure_lt`	Phase, semantic rank, and potential are lexicographically ordered.
Proof certificate	`Cyclic.certificate`	Stores executable data and dependent semantic evidence.
Reflection	`certify_raw_check`	Projects structure and witness success into certificate fields.
Bounded theorem search	`bounded_graph_search_from`, `search_depths`	First-open-node DFS followed by iterative deepening.
Successful Boolean query	`solvedb`; theorem `search_sound`	Only the true direction has a validity theorem.

Appendix B

Module and symbol index

Table B.1: All SP modules and the necessary Cyclic modules.

Module	Symbols and role
Cyclic/Relations.v	slope, arc, relation_compose, relation_closure, path_relation, path_closure, relational_trace_check. Executable two-slope relation and graph-path closure algebra.
Cyclic/Framework.v	THEORY, Make, Valid, edge_info, respects_edge, rule_instance, axiom_instance, graph, graph builders, structural_check, relational_check, trace_witness, witness_check, raw_check, certificate, certificate_step_decreases, certificate_sound.
SP/Syntax.v	Reified variables, terms, tags, constraints, pure and spatial atoms, formulas, clauses, definitions, judgments, decidable equality, tag erasure, substitution, matching primitives, freshening, identity normalization, inconsistency checks, visible tags, lookup, and constructors.
SP/Semantics.v	concrete_heap, heap_wf, heap_join, valuations, exact points-to, spatial_holds_with, predicate_holds_rank, tag bounds, constraints, formula_holds, world, denote, SeparationLogicTheory, and the local Cyclic functor instance.
SP/Rules.v	Identity, tag erasure, explicit ex falso, common-rank lemmas, predicate introduction, left disjunction, right choice, exact backlink, and the independently-valid progressing teaching rule.
SP/Automation.v	Strict self-constraint rejection, existential-substitution check, fresh identifiers, and identity normalization equivalence.
SP/Certification.v	Structural reflection, countermodel advance, false-node liveness, witness reflection, certify_raw_check, raw_check_sound, and uniform one-node axiom certificates.

Module	Symbols and role
SP/RuleGeneration.v	Ordered one-way heap matcher; identity, inconsistency, erasure, and predicate-introduction axiom candidates; left and right disjunction rule candidates; named unsupported stubs.
SP/GraphSearch.v	First-open-node selection, successor priority, backlink enumeration, tag/potential witness enumeration, checked_graph, bounded DFS, and searched-graph certification.
SP/Definitions.v	Hand-reified predicate libraries: sl_definitions, atva_definitions, and songbird_definitions; 38 entries in the main library.
SP/Benchmarks.v	Nine selected reified entailments and selected_benchmarks, including benchmark_base_19_1.
SP/Examples.v	Genuine acyclic identity certificate; artificial progressing two-node teaching graph; rejected open, self-backlink, and non-progressing graphs; printed assumptions.
SP/Tests.v	Positive and negative generator equations plus normalization and exact points-to regression examples.
SP/Search.v	search_config, verified_proof, verified_proof_valid, generated graph strategy, iterative deepening, solvedb, search_sound, genuine search computations, and benchmark_base_19_search_valid.

Appendix C

End-to-end audit checklist

When reading a claimed success in this codebase, the following compact audit reconstructs the theorem path without replaying every proof.

1. Identify the exact `judgment`: definition environment, antecedent, succedent, tags, and constraints.
2. Determine which candidate wins `generate_axiom`; if none wins, list the backlinks and `generate_rules` candidates in their real order.
3. For every installed rule, inspect both `rule_theorem` and `rule_counter_step`; for every axiom, inspect `axiom_theorem`.
4. Check whether the graph is acyclic, a genuine progressing cycle, or an artificial cycle built from an independently valid judgment.
5. Expand `raw_check` into structure, relational closure, and witness checks. Record the actual witness candidate that succeeds.
6. Follow `certify_raw_check`: structure gives root, liveness, and advance; the witness check gives trace transitions; the relational check is an additional acceptance condition.
7. Apply `certificate_sound` and then the root equality stored in `verified_proof`. Only here state semantic validity of the input goal.
8. If computation returns `None` or `solvedb = false`, report the depth interval and say only "bounded non-success."
9. Run `Print Assumptions` on the exported theorem and compare with Chapter 13.

For `benchmark_base_19_search_valid`, this audit collapses to a particularly clean chain:

$$\begin{aligned}
&\text{recursive } \mathsf{Is} \text{ clause matches the antecedent}\\
&\Longrightarrow \text{theorem-backed predicate-introduction axiom}\\
&\Longrightarrow \text{one-node closed graph}\\
&\Longrightarrow \text{empty witness and vacuous loop checks}\\
&\Longrightarrow \text{reflected semantic certificate}\\
&\Longrightarrow \mathsf{Valid}(\texttt{benchmark_base_19_1}).
\end{aligned}$$

Conclusion

The `SP` development is a compact demonstration of proof-producing automation. Its heap semantics is exact and ranked; its rule packages contain both forward validity and countermodel propagation; its finite graphs are checked for structure and two forms of global progress; and its reflection layer converts a successful computation into the semantic evidence required by a generic lexicographic infinite-descent theorem.

The most important negative fact is equally clear: the current search calculus is intentionally incomplete. Identity normalization, ordered matching, one right choice, exact non-progressing backlinks, and explicit `None` stubs leave many valid entailments outside its reach. That boundary is safe because the public API proves only the success direction. The result is a useful base for theorem-first extension: add semantic lemmas, package a locally sound rule with a countermodel step, expose it through a conservative generator, and let the existing graph-to-certificate path do the rest.

Bibliography

[1] John C. Reynolds. Separation logic: A logic for shared mutable data structures. In *Proceedings of the 17th Annual IEEE Symposium on Logic in Computer Science (LICS 2002)*, pages 55–74. IEEE Computer Society, 2002. doi:10.1109/LICS.2002.1029817.

[2] James Brotherston, Dino Distefano, and Rasmus Lerchedahl Petersen. Automated cyclic entailment proofs in separation logic. In Nikolaj Bjørner and Viorica Sofronie-Stokkermans, editors, *Automated Deduction – CADE-23*, volume 6803 of *Lecture Notes in Computer Science*, pages 131–146. Springer, 2011. doi:10.1007/978-3-642-22438-6_12.

[3] James Brotherston and Alex Simpson. Sequent calculi for induction and infinite descent. *Journal of Logic and Computation*, 21(6):1177–1216, 2011. doi:10.1093/logcom/exq052.

[4] James Brotherston, Nikos Gorogiannis, and Rasmus Lerchedahl Petersen. A generic cyclic theorem prover. In Ranjit Jhala and Atsushi Igarashi, editors, *Programming Languages and Systems: 10th Asian Symposium, APLAS 2012*, volume 7705 of *Lecture Notes in Computer Science*, pages 350–367. Springer, 2012. doi:10.1007/978-3-642-35182-2_25.